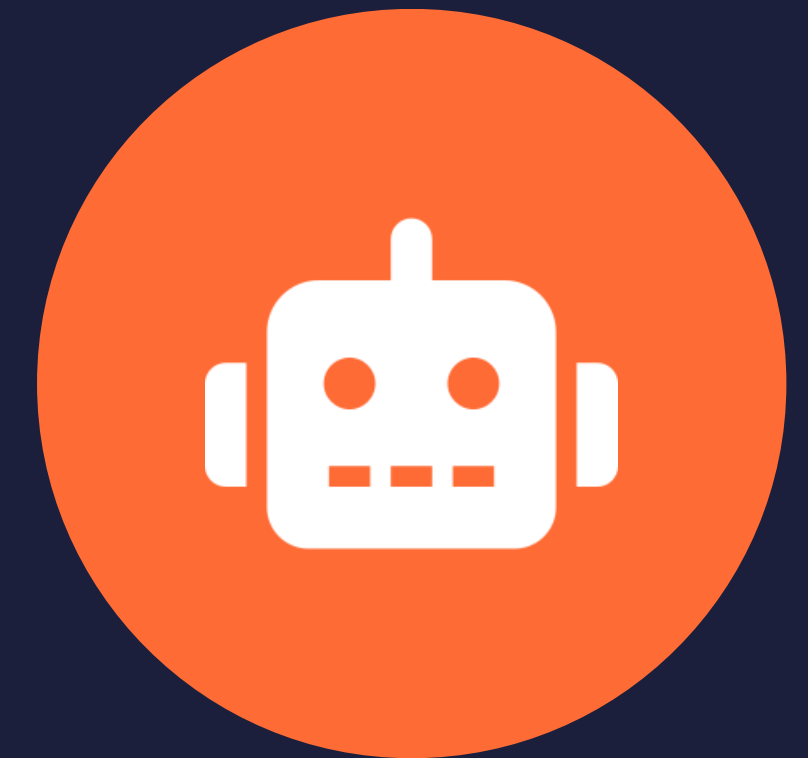


IME III · PRIMER PARCIAL

Sistema de Gestión de Competencia de Robótica

Diseño orientado a objetos en C++



El problema



- Los equipos inscriben robots que compiten por categorías (disciplinas).
- Hoy todo se gestiona “a mano”: se pierden datos y no hay trazabilidad.
- Se necesita un sistema que registre equipos, integrantes y robots, y organice los enfrentamientos.



Antes

Registros en papel, propensos a error y difíciles de consultar.



Después

Un sistema que centraliza equipos, robots y resultados de las batallas.

Objetivo del proyecto



Identificar objetos

Traducir el problema real a clases con datos y comportamientos.



Encapsulamiento y mensajes

Los objetos colaboran enviándose mensajes entre sí.

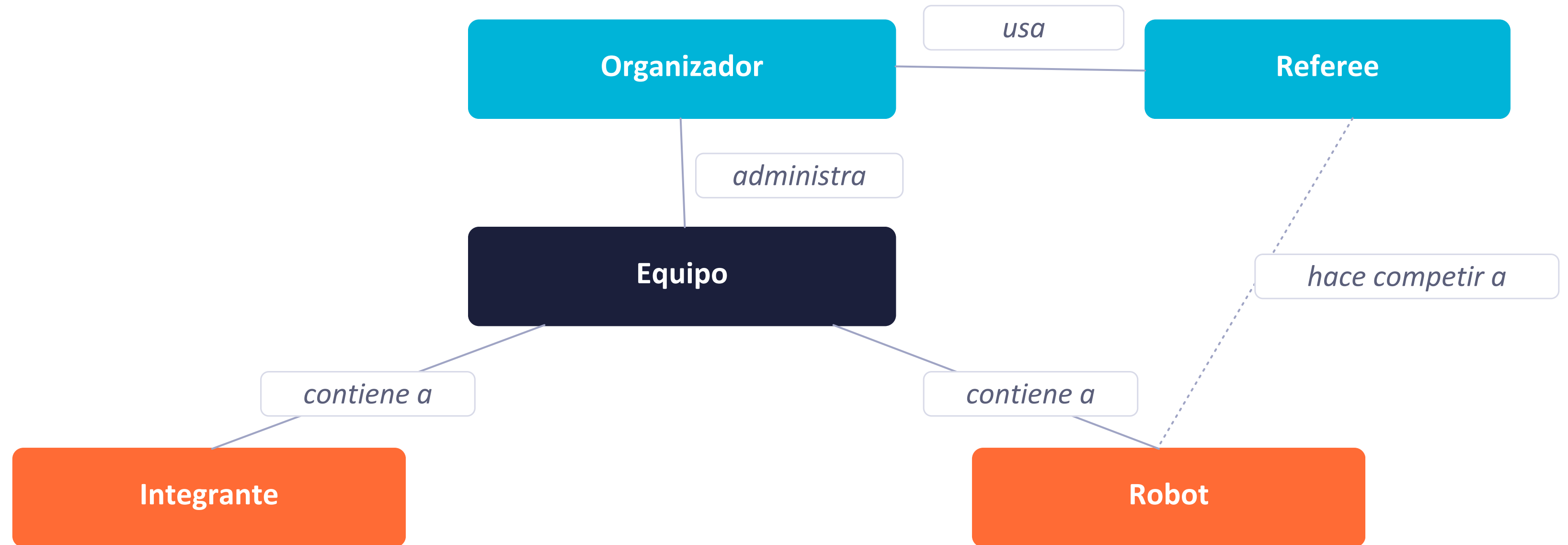


Ciclo de vida completo

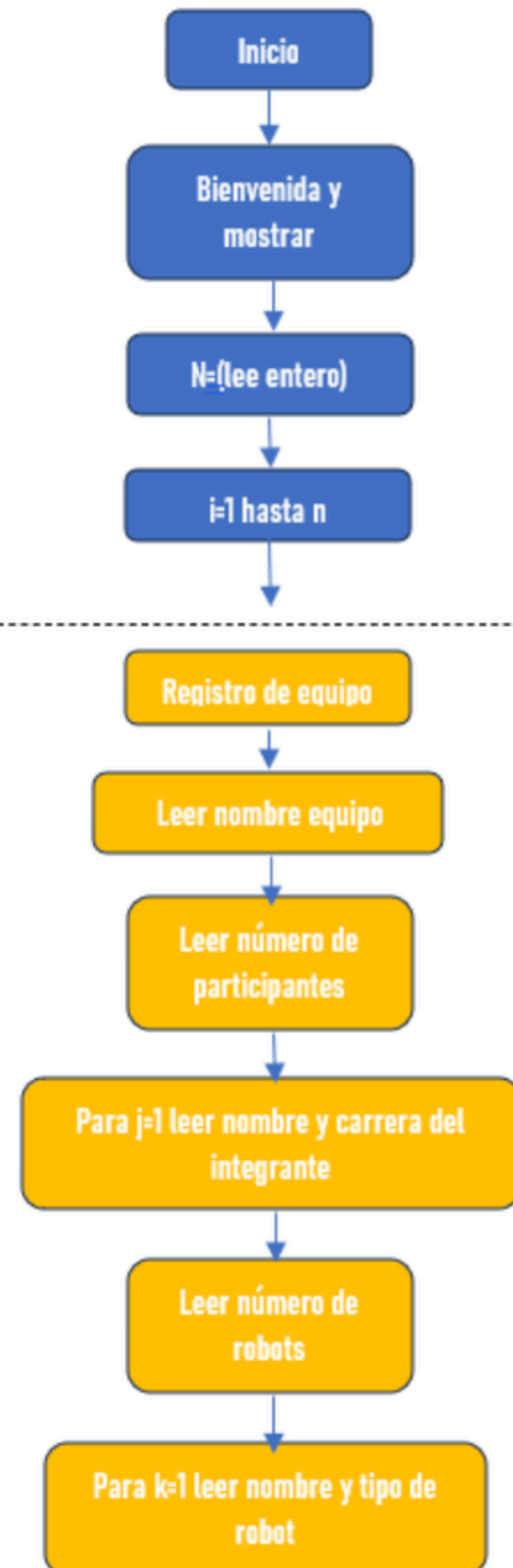
Análisis, diseño, código, pruebas y documentación.

Arquitectura de clases

Objetos que colaboran mediante mensajes



Flujo del registro



1

Leer cuántos equipos se registrarán (n).

2

Por cada equipo: leer nombre, integrantes (nombre y rol) y robots (nombre y tipo).

3

Cada robot se inscribe automáticamente en su disciplina, según su tipo.

4

Al terminar los n equipos, se cierra el registro y se muestra el resumen.

Cómo se registra un equipo

```
Equipo equipo(nombreEq);

for (int j = 0; j < numInt; ++j) {
    getline(cin, nomInt);
    getline(cin, rolInt);
    equipo.agregarIntegrante(
        Integrante(nomInt, rolInt));
}

for (int k = 0; k < numRob; ++k) {
    getline(cin, nomRob);
    getline(cin, tipoRob);
    equipo.agregarRobot(
        Robot(nomRob, tipoRob));
}

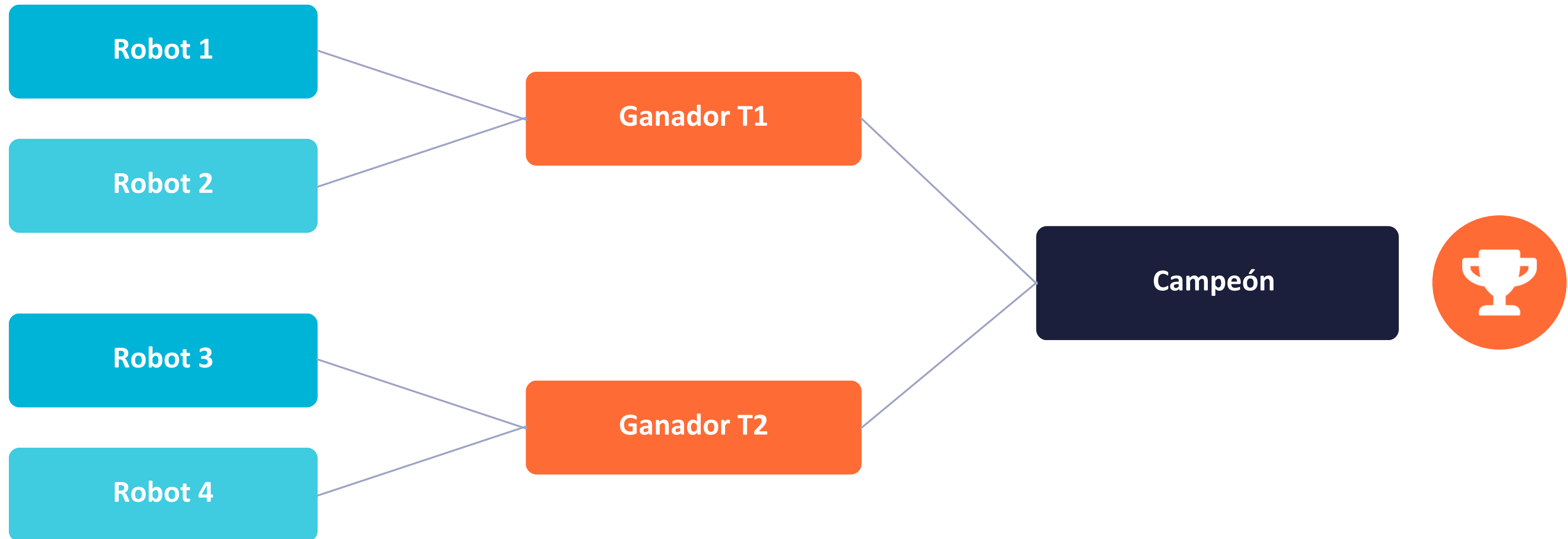
equipos.push_back(equipo);
```

- Cada equipo es un objeto Equipo que agrupa a sus integrantes y robots.
- agregarIntegrante() y agregarRobot() son mensajes: el Equipo no sabe cómo se leyeron los datos, solo los recibe.
- El constructor de Robot convierte el texto del tipo a la categoría interna (enum Categorías) automáticamente.

¿Quién gana? Referee decide al azar



Un torneo de eliminación por disciplina



Decisiones de diseño: casos especiales



Una disciplina sin robots registrados se omite (continue) — no genera batallas.



Un robot que queda solo en su disciplina se declara ganador automático, sin batalla.



El orden de los enfrentamientos sigue el orden de registro, no un sorteo adicional.

Aprendizajes y próximas mejoras



Aprendizajes

- Diseñar antes de programar ahorra tiempo
- Es mejor usar enum class que string
- Dividir el trabajo en equipo por responsabilidades claras.



Mejoras futuras

- Guardar datos en archivos
- Validar mejor lo que escribe el usuario

